Visualizing Lexer Stages: From NFA to DFA

Willkommen in diesem Projekt!
Dieses dokumentiert alle Phasen des Projekts – von der Vision und den Anforderungen bis hin zur Implementierung, den Tests und der Planung.

Inhaltsverzeichnis

1. Vision und Kontext
2. Systemanforderungen
3. User Stories
4. Tests und Verifikation
5. Sprints und Zeitplan
6. Architektur und Design

Team

- **Dateng Tankoua Emery Josian**
  Datengta@students.uni-marburg.de
  Bachelorstudent, Fachbereich Mathematik und Informatik
  Rolle: Software-Engineering Team
- **Zhang Ziyan**
  Zhangzi@students.uni-marburg.de
  Masterstudentin, Fachbereich Mathematik und Informatik
  Rolle: Software-Engineering Team
- **Betreuerin:**
  M.Sc. Teresa Dreyer
  dreyert@staff.uni-marburg.de
  Rolle: Betreuung, Feedback, Abnahme

Projektübersicht

Das Ziel des Projekts ist die Entwicklung eines webbasierten Tools, das die schrittweise Umwandlung eines **nichtdeterministischen endlichen Automaten (NFA)** in einen **deterministischen endlichen Automaten (DFA)** visualisiert.
Die Umsetzung erfolgt mit **React**, **TypeScript**, **ReactFlow**, **D3.js** und **shadcn/ui** unter Anwendung des **MVC-Architekturmodells** und eines **iterativen (agilen) Entwicklungsprozesses**.

Vision und Kontext

Hintergrund

Im Rahmen des Moduls „Programming Languages and Tools“ an der Philipps-Universität Marburg soll ein Visualisierungstool zur Veranschaulichung des Lexers-Prozesses entwickelt werden.

Motivation

Studierende sollen ein besseres Verständnis für den Übergang vom NFA zum DFA gewinnen, der eine zentrale Rolle bei der lexikalischen Analyse in Compilerbau spielt.

Projektziel

Das Tool soll:

- eine Eingabe in einer domänenspezifischen Sprache (DSL) einlesen,
- den Umwandlungsprozess NFA → DFA schrittweise ausführen,
- jede Phase visuell darstellen,
- das Ergebnis wieder in derselben DSL exportieren.

Beteiligte Personen

Name	Rolle	E-Mail
Dateng Tankoua Emery Josian	Student / Entwickler	Datengta@students.uni-marburg.de
Zhang Ziyan	Studentin / Entwicklerin	Zhangzi@students.uni-marburg.de
M.Sc. Teresa Dreyer	Betreuerin	dreyert@staff.uni-marburg.de

Technologie-Stack

- React + TypeScript
- ReactFlow + D3.js
- shadcn/ui
- GitLab (Versionierung, Wiki, Issues, CI/CD)
- VS Code (Entwicklungsumgebung)

Vorgehensmodell

Der Entwicklungsprozess folgt einem **iterativen/agilen Modell (Iteration Process)** mit kurzen Sprints, regelmäßigen Reviews und klar definierten Milestones.

Systemanforderungen

ID	Beschreibung	Status	Verknüpfte Issues
REQ-01	Das System soll den Import eines NFAs in DSL-Format ermöglichen.	Offen	#3
REQ-02	Der Algorithmus zur Subset-Construction (NFA → DFA) soll korrekt implementiert werden.	Offen	#4
REQ-03	Jede Umwandlungsstufe soll visuell dargestellt werden können.	Offen	#5
REQ-04	Der Benutzer soll zwischen den Schritten navigieren können (Start, Zurück, Weiter, Ende).	Offen	#6
REQ-05	Das resultierende DFA soll im DSL-Format exportiert werden können.	Offen	#7
REQ-06	Die Anwendung soll vollständig lokal (ohne Server) in React + TypeScript laufen.	Offen	#8
REQ-07	Es sollen Unit- und Integrationstests für den Algorithmus existieren.	Offen	#9

Hinweis: Jede Anforderung ist mit einer oder mehreren User Stories und Tests verknüpft.

User Stories

ID	Als ...	möchte ich ...	damit ...	Zugehörige REQs	GitLab-Issue
US-01	Nutzer	eine DSL-Datei importieren	ich den NFA sehen kann	REQ-01	#3
US-02	Nutzer	den NFA grafisch angezeigt bekommen	ich die Zustände und Übergänge verstehe	REQ-01, REQ-03	#4
US-03	Student	den Subset-Construction-Algorithmus Schritt für Schritt durchlaufen	ich die Bildung des DFA nachvollziehen kann	REQ-02, REQ-03	#5
US-04	Nutzer	zwischen den Schritten navigieren (Start, Zurück, Weiter, Ende)	ich flexibel zwischen Zuständen wechseln kann	REQ-04	#6

ID	Als ...	möchte ich ...	damit ...	Zugehörige REQs	GitLab-Issue
US-05	Nutzer	den finalen DFA exportieren	ich das Ergebnis weiterverwenden kann	REQ-05	#7
US-06	Entwickler	automatisierte Tests ausführen	die Korrektheit des Algorithmus sichergestellt ist	REQ-07	#9

Tests und Verifikation

Test-ID	Beschreibung	Verknüpfte REQs	Testtyp	Status
TEST-01	Import einer Beispiel-DSL und Anzeige des NFA	REQ-01	Manuell / Integration	Offen
TEST-02	Prüfung der korrekten Berechnung von Subset-Construction	REQ-02	Unit-Test	Offen
TEST-03	Kontrolle, dass der DFA die gleiche Sprache erkennt wie der NFA	REQ-02, REQ-05	Integration	Offen
TEST-04	Navigation zwischen Schritten (Prev, Next, Start, End) funktioniert	REQ-04	Manuell / UI-Test	Offen
TEST-05	Export des DFA im DSL-Format korrekt	REQ-05	Unit / Integration	Offen
TEST-06	Alle Unit-Tests bestehen	REQ-07	Automatisiert	Offen

Sprints und Zeitplan

Sprint 0 – Vorbereitung (Woche 1)

- Einrichtung des GitLab-Repositories
- Hinzufügen der Teammitglieder (Emery, Ziyan, Teresa)
- Setup von React, TypeScript, CI/CD
- Anlegen der Wiki-Struktur **Ziel:** funktionierendes Grundgerüst

Sprint 1 – NFA Import & Visualisierung (Woche 2–3)

- US-01, US-02
- DSL-Parser implementieren
- Erste Darstellung des NFA mit ReactFlow **Deliverable:** Sichtbarer NFA aus DSL

Sprint 2 – Subset-Construction-Algorithmus (Woche 4–5)

- US-03, US-04
- Implementierung und schrittweise Simulation **Deliverable:** Schritt-für-Schritt-Visualisierung

Sprint 3 – Export & Feinschliff (Woche 6–7)

- US-05, US-06
- Export-Funktion, Unit-Tests, UI-Optimierung **Deliverable:** stabile, exportierbare Version

Sprint 4 – Dokumentation & Präsentation (Woche 8)

- Fertigstellung der Wiki-Dokumentation
- Vorbereitung der Demonstration für die Betreuung **Deliverable:** Präsentationsreife Version

---

Aufgabenverteilung

Person	Hauptaufgaben
Dateng Tankoua Emery Josian	Frontend, UI, Visualisierung, Dokumentation
Zhang Ziyan	Algorithmus, Tests, Logik, Export
Teresa Dreyer	Betreuung, Feedback, Abnahme

Architektur und Design

Architekturmodell

Das Projekt folgt dem **MVC-Muster (Model-View-Controller)**.

- **Model:**
  Repräsentation des NFA, DFA und der Zwischenschritte.
  Enthalten in `/src/core/` (types.ts, subsetConstruction.ts)
- **View:**
  Darstellung der Automaten und Übergänge.
  Implementiert mit **ReactFlow** und **D3.js** in `/src/components/GraphViewer/`.

- **Controller:**
  Steuerung der Schritt-Navigation (Vor, Zurück, Start, Ende).
  Verantwortlich für den Zustand der Simulation.

Projektstruktur

```
src/
 ├── core/                  # Modelle & Algorithmus (NFA, DFA, Subset-
Construction)
 ├── components/            # UI-Komponenten (Grafen, Buttons, Panels)
 ├── pages/                 # App-Seiten
 ├── utils/                 # Parser & Helper-Funktionen
 ├── App.tsx
 └── main.tsx
```